UNIT-IV

**Three Dimensional Viewing:** Introduction, Representation of Three-dimensional objects, Projections, Parallel projections: Orthographic Projections, Oblique Projections. Perspective Projection, Three dimensional clipping, Three-dimensional Cohen-Sutherland clipping algorithm.
Hidden Surface Removal: Depth-Buffer(z-buffer) method, Depth-sorting Method(Painter's algorithm)

**3-D Clipping**

Clipping is the process of removing objects that are not visible from the scene. The clipping process helps in reducing the computational effort. It can be obtained by following two steps:
1. Deleting objects that cannot be viewed. e.g., objects behind camera, outside the view field or that are too far from sight.
2. Also remove the object that intersects with the clipping plane.

**Text clipping**

Text can be represented through clipping. With respect to text, clipping can be divided into area and line clipping. Area clipping comes into picture when bitmaps are used to represent characters. When characters are represented by lines, line clipping is implemented. But in general cases, characters are a combination of lines and curves. Hence, they are called vectors.
Clipping of a text can be categorized in the following ways:
1. Precise clipping
2. Clipping to a character
3. Clipping a text by words

Each text is seen as an inseparable object in the case of clipping. Every character consists of a rectangle that is called as "rectangle envelope". The center of the envelope is tested in the display box while clipping two characters to each other.

The purpose of 3D clipping is to identify and save all surface segments within the view volume for display on the output device. All parts of objects that are outside the view volume are discarded. Thus the computing time is saved.
3D clipping is based on 2D clipping. To understand the basic concept we consider the following algorithm:

**Polygon Clipping**

Assuming the clip region is a rectangular area,
1. The rectangular clip region can be represented by xmin, xmax, ymin and ymax.
2. Find the bounding box for the polygon: ie. the smallest rectangle enclosing the entire polygon.
3. Compare the bounding box with the clip region (by comparing their xmin, xmax, ymin and ymax).
4. If the bounding box for the polygon is completely outside the clip region (case 2), the polygon is outside the clip region and no clipping is needed.
5. If the bounding box for the polygon is completely inside the clip region (case 1), the polygon is inside the clip region and no clipping is needed.
6. Otherwise, the bounding box for the polygon overlaps with the clip region (cases 3 and 4) and the polygon is likely to be partly inside and partly outside of the clip region. In that case, we clip the polygon against each of the 4 border lines of the clip region in sequence as follows:

1. Using the first vertex as the current vertex. If the point is in the inside of the border line, mark it as 'inside'. If it is outside,

mark it as 'outside'.

2. Check the next vertex. Again mark it 'inside' or 'outside' accordingly.

3. Compare the current and the next vertices. If one is marked 'inside' and the other 'outside', the edge joining the 2 vertices crosses the border line.

4. In this case, we need to calculate where the edge intersects the border (ie. intersection between 2 lines).

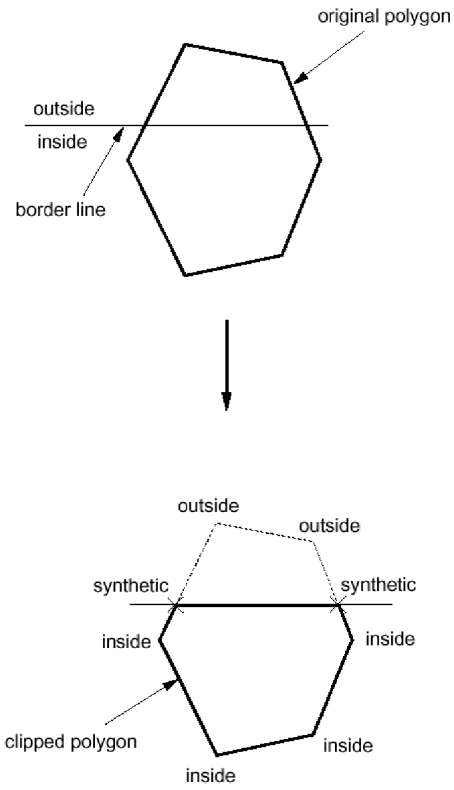
5. The intersection point becomes a new vertex and we mark it as 'synthetic'.

6. Now we set the next vertex as the current vertex and the following vertex as the next vertex, and we repeat the same operations until all the edges of the polygon have been considered.

7. After the whole polygon has been clipped by a border, we throw away all the vertices marked 'outside' while keeping those marked as 'inside' or 'synthetic' to create a new polygon.

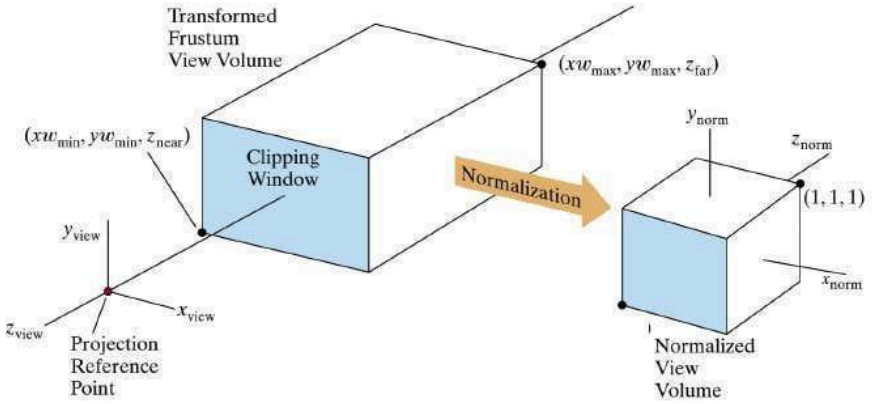
8. We repeat the clipping process with the new polygon against the next border line of the clip region.

7. This clipping operation results in a polygon which is totally inside the clip region.



Because of the extraordinary computational efforts required, two types of clipping strategies are followed:
– Direct Clipping: The clipping is done directly against the view volume.
– Canonical Clipping: Normalization transformations areapplied which transform the original view volume into normalized (canonical) view volume. Clipping is then performed against canonical view volume.

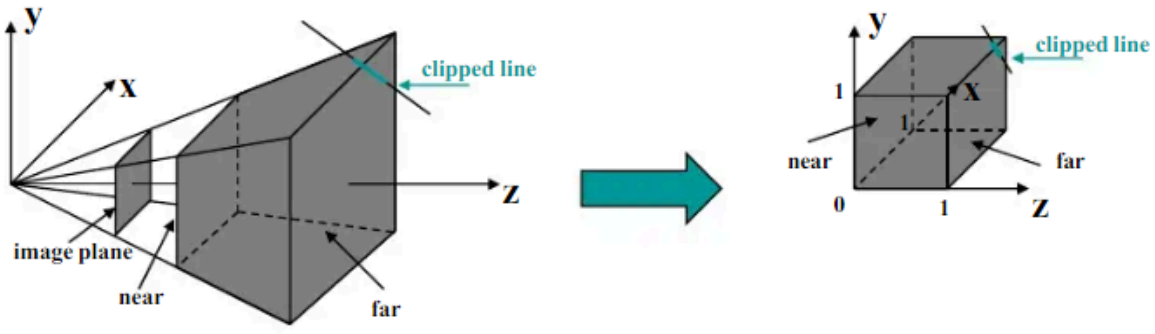
Clipping Strategies
• The canonical view volume for parallel projection is the unit cube whose faces are defined by planes
x = 0 ; x = 1 y = 0; y = 1 z = 0; z = 1




Clipping Strategies
•The canonical view volume for

perspective projection is the truncated normalized pyramid whose faces are defined by planes
x = z ; x = -z y = z; y = -z z = $z_f$ ; z = 1

$x = z\ ;\ x = -z$ $y = z;\ y = -z$ $z = z_f;\ z = 1$




**Clipping Strategies**
• The basis of canonical clipping is the fact that intersection of line segments with the faces of canonical view volume require minimal calculations.

• For perspective views, additional clipping may be required toavoid perspective anomalies produced by the projecting objects that are behind view point.

**3-D Viewing**

The 3-D viewing process is much more complex than the 2-D viewing operations. Here projections are implemented to convert 3-D objects into 2-D projection planes. Here, the view volume is specified in the world coordinates using the modeling transformation. These transformations are next used in the conversion process.

In 3-D viewing, a viewing volume specifies a viewing volume with a projection method in the world coordinates, and a viewport on the display.

1. Viewing volume states what to appear
2. View port specifies where to display

For viewing an object, the camera and perspective projection is required. The camera properties are the following:

1. Eye at a point in space
2. View volume whose apex is at the eye
3. Field of view θ
4. Near and far clipping images
5. View plane and its aspect ratio

**The Viewing Pipeline**

The 3-D viewing volume is processed for projection methods. Objects are clipped against the 3-D viewing volume. The contents are then transformed into the viewport for display. The 3-D coordinates are preserved for lighting and hidden-surface removal.

**Graphic Pipeline in 3-D Viewing**

In the case of 3-D viewing, a view volume is described in terms of world coordinates. The modeling transformation is used for the same. The positions of world coordinates are transformed to viewing coordinates with the implementation of viewing transformation. The description in viewing coordinates is converted to 2-D projection coordinates using the projection transformation. In the final stage, the projection coordinates are transformed into device coordinates through the workstation transformation. The figure depicts the pipeline.

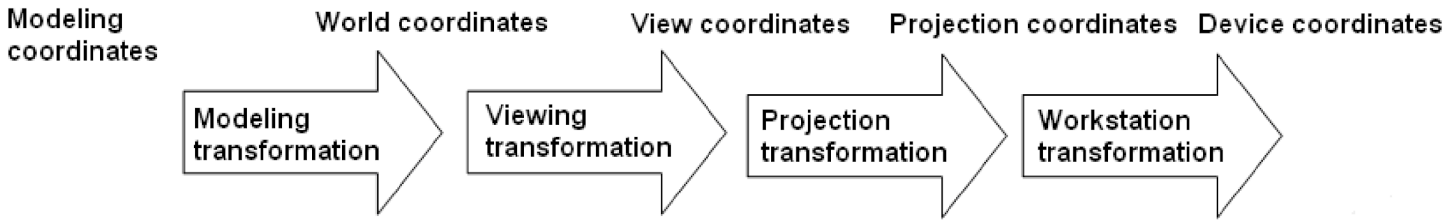


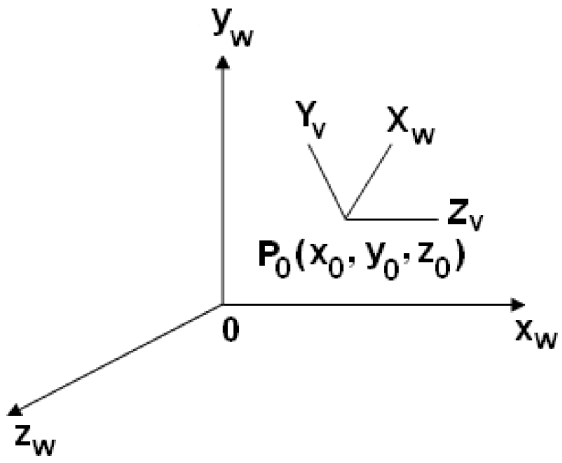

**Figure : 3-D Viewing Pipeline**

The stages in the graphic pipeline are responsible for information processing that are provided at initial

stages as just properties at the vertices or control points. They specify what has to be rendered. The basic objects related to 3-D graphics are lines and triangles. They include texture, reflectivity, RGB values, etc.

**Camera Coordinates**

A view plane is a film plane in a camera that is positioned for a particular shot of the scene. The world position coordinates are converted into viewing coordinates and projected into the view plane. The view plane can be established by viewing coordinate system or View Reference Coordinates (VRC).




**Viewing Coordinate System**

Here the first view parameter to be considered is the V**iew Reference Point** [VRP]. It is the center of view reference system. It is normally close to or on the surface of some object in a scene.

The second viewing parameter is the **View-Plane Normal vector** [VPN]. The normal vector is the direction that is perpendicular to the view plane.

The **view distance** is another parameter. The normal vector is a directed line segment from view plane to view reference point. The length of the directed line segment is called the view distance. It is illustrated in the figure 8.14. These parameters allow the use to select the necessary object view.

**Transforming World Coordinates to Viewing Coordinates**

The position of objects can be described in different ways namely, the camera coordinates and the world coordinates. The world coordinates can be converted to viewing coordinates by the following procedure:

1. To translate view reference point to origin of world coordinate system.
2. To align the xv, yv, zv axes with the world coordinates xw, yw and zw axes, apply the rotations.

$$T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -x_p & -y_p & -z_p & 1 \end{pmatrix}$$

P’ =P.T

To align the three axes, the three coordinate-axis rotations are required. It depends on the direction that is chosen for N. Here, N is the normal vector.

**Clipping Algorithms**

**3D clipping algorithms are direct adaptation of 2D clipping algorithms with following modifications:**

1.For Cohen Sutherland :

2.For Liang-Barsky: Introduction of new equations

3.For Sutherland Hodgeman: Inside/Out side Test

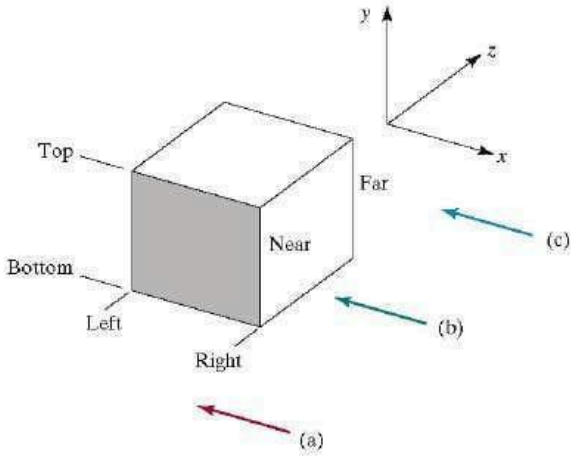
3D Cohen-Sutherland Line Clipping

•Similar to the case in two dimensions, we divide the world into regions

• This time we use a 6-bit region code to give us 27 differentregion codes

• The bits in these regions codes are as follows:

| bit1 | bit2 | bit3 | bit 4 | bit 5 | bit 6 |
|---|---|---|---|---|---|
| Top | Bottom | Right | | Left | Behind | Front |




| 011001 | 011000 | 011010 |
|---|---|---|
| 010001 | 010000 | 010010 |
| 010101 | 010100 | 010110 |

Region Codes
In Front of Near Plane
(a)

| 001001 | 001000 | 001010 |
|---|---|---|
| 000001 | 000000 | 000010 |
| 000101 | 000100 | 000110 |

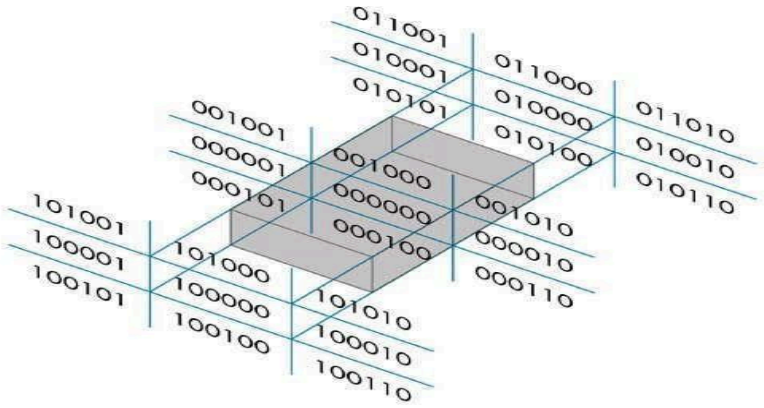
Region Codes
Between Near and Far Planes
(b)

| 101001 | 101000 | 101010 |
|---|---|---|
| 100001 | 100000 | 100010 |
| 100101 | 100100 | 100110 |

Region Codes
Behind Far Plane
(c)

**3D Cohen-Sutherland Line Clipping**

Now we use a 6 bit out code to handle thenear and far plane.The testing strategy isvirtually identical tothe 2D case.




# 3D Cohen-Sutherland Line Clipping

**CASE – I** Assigning region codes to endpoints for *Canonical Parallel View Volume* defined by:

$x = 0\ ,\ x = 1;$ $\quad y = 0,\ y = 1;$ $\quad z = 0,\ z = 1$

The bit codes can be set to true(1) or false(0) for depending on the test for these equations as follows:

```
Bit 1 ≡ endpoint is Above view volume = sign (y-1)
Bit 2 ≡ endpoint is Below view volume = sign (-y)
Bit 3 ≡ endpoint is Right view volume = sign (x-1)
Bit 4 ≡ endpoint is Left view volume = sign (-x)
Bit 5 ≡ endpoint is Behind view volume = sign (z-1)
Bit 6 ≡ endpoint is Front view volume = sign (-z)
```

- To clip lines we first label all end points with the appropriate region codes.
- Classify the category of the Line segment as follows
  - *Visible*: if both end points are 000000
  - *Invisible*: if the bitwise logical AND is not 000000
  - *Clipping Candidate*: if the bitwise logical AND is 000000
- We can trivially accept all lines with both end-points in the [000000] region.
- We can trivially reject all lines whose end points share a common bit in any position.

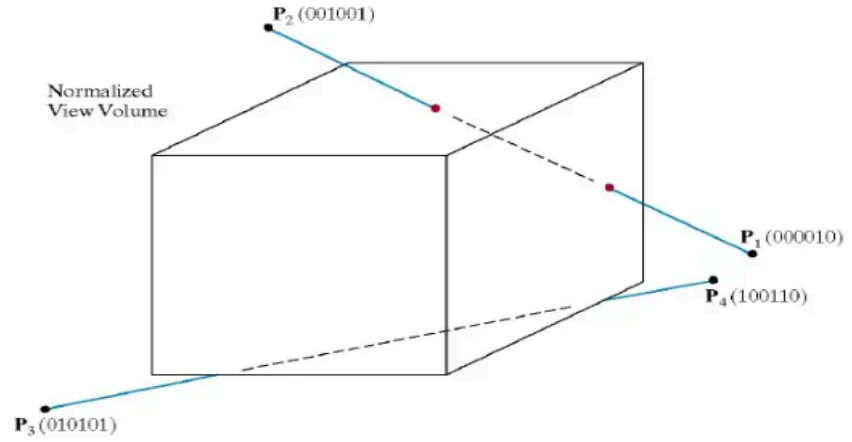
- For clipping equations for three dimensional line segments are given in their parametric form.
- For a line segment with end points $P_1(x1_h, y1_h, z1_h, h1)$ and $P_2(x2_h, y2_h, z2_h, h2)$ the parametric equation describing any point on the line is:

$$P = P_1 + (P_2 - P_1)u \qquad 0 \le u \le 1$$

- From this parametric equation of a line we can generate the equations for the homogeneous coordinates:

$$x_h = x1_h + (x2_h - x1_h)u$$
$$y_h = y1_h + (y2_h - y1_h)u$$
$$z_h = z1_h + (z2_h - z1_h)u$$
$$h = h1 + (h2 - h1)u$$

- Consider the line $P_1[000010]$ to $P_2[001001]$
- Because the lines have different values in bit 2 we know the line crosses the right boundary






## 3D Cohen-Sutherland Line Clipping

- Since the right boundary is at $x = 1$ we now know the following holds:

$$x_p = \frac{x_h}{h} = \frac{x1_h + (x2_h - x1_h)u}{h1 + (h2 - h1)u} = 1$$

- which we can solve for $u$ as follows:

$$u = \frac{x1_h - h1}{(x1_h - h1) - (x2_h - h2)}$$

- using this value for $u$ we can then solve for $y_p$ and $z_p$ similarly
- Then simply continue as per the two dimensional line clipping algorithm

**Viewing Volume**

An orthographic projection is a parallel projection if the projection lines are all normal to the view plane. The orthographic viewing volume constitutes the six plane equations, x=left, x=right, y=top, y=bottom, and z=-near or far.

The camera is always set in such a way that the eye is at the origin and the VPN is aligned with the zaxis. The look point defined by the programmer acts as a point of importance in the scene, and when combined with eye, the look and eye together defines the View Plane Normal (VPN) as eye-look.

**Setting the View Volume**

To view a scene the camera is moved and aimed in a particular direction. The viewing is done by performing the rotation and translation processes, it is part of model view matrix that stores the exact vector data that defines the three directions.

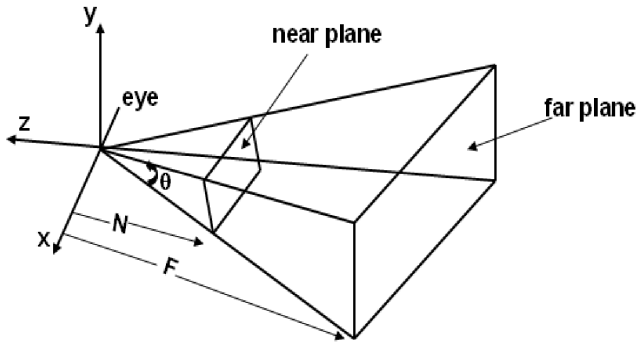
By setting the view volume, the camera is moved so that its eye is positioned at point eye. The position

of look is towards the point of importance. The vector up suggests the upward direction, which is in normal case set to (0, 1, 0).

Hence, the view volume defines the volume in 3-D. When the object is projected, the view volume assures that the projection fits the viewport. The figure 8.13 clearly shows the two planes. The view volume can be divided into two parts namely,

*View Plane Rectangle*: It defines the boundaries of x and y plane that will be mapped into the viewport. Rendering is possible only to the objects that lie inside the rectangle view plane.

*Near Far Clipping*: The bounds for object are defined only for the z direction in the near far clipping planes. Rendering here is possible only for the objects that lie nearer to the eye than the near clipping plane and also for the objects that lie farther from the eye and far from the clipping plane.

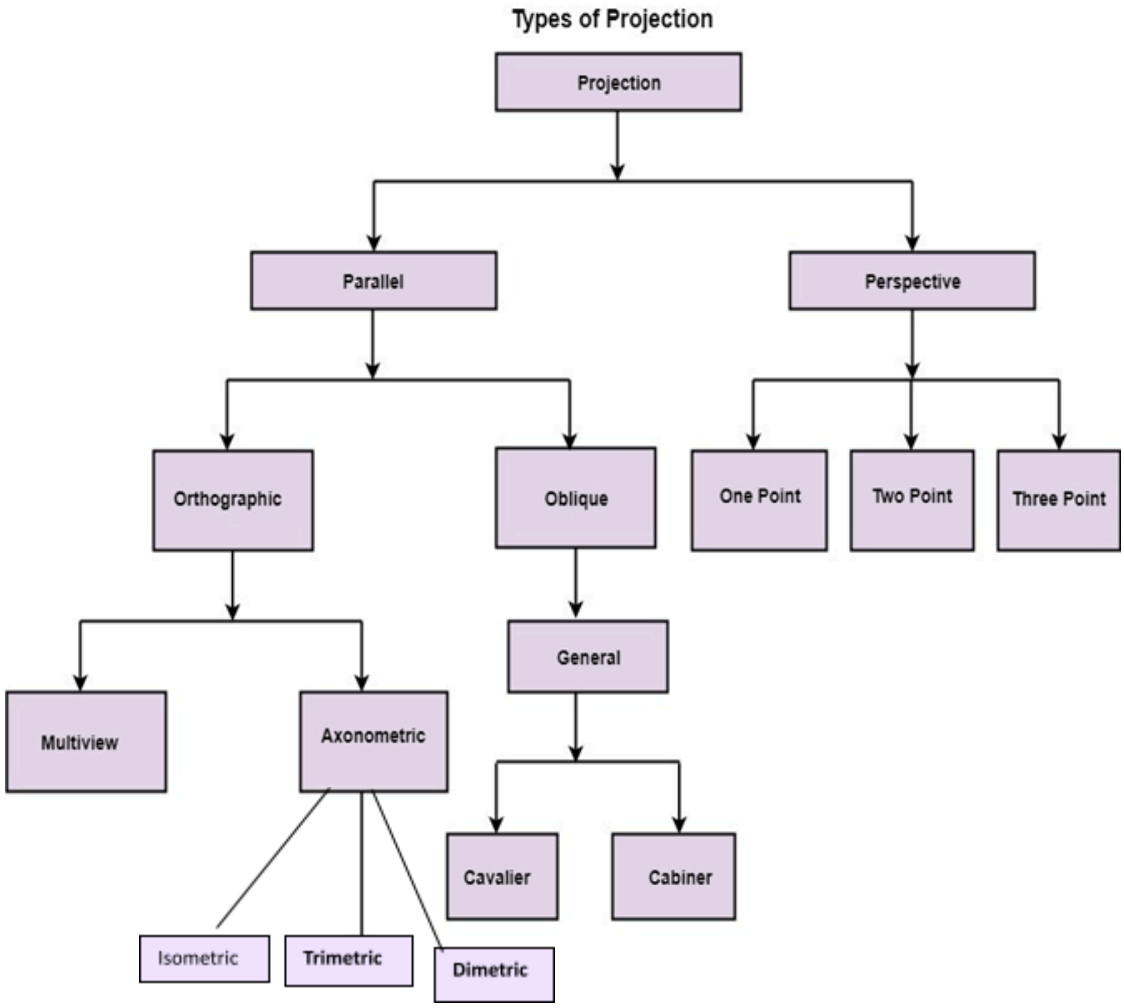



**Specifying an Arbitrary 3-D View**

We know that an object can be viewed from any direction. It is also necessary to define a view plane based on a specified point of view. The figure 8.12 displays the view plane and the view coordinate system.

**Projection**

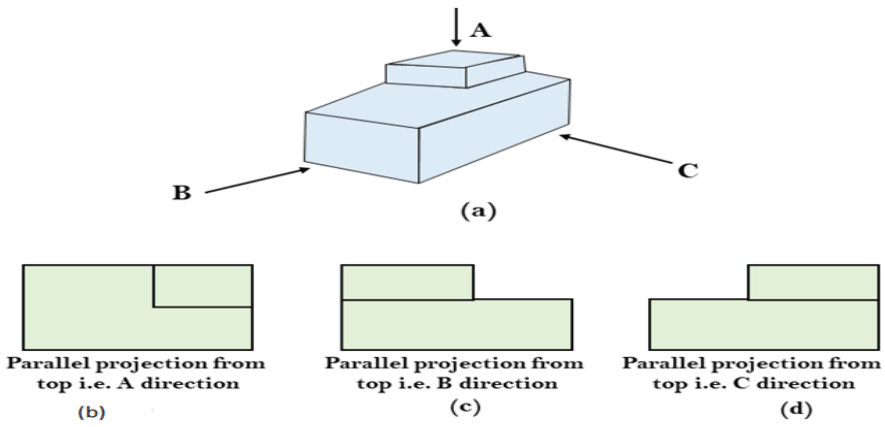
It is the process of converting a 3D object into a 2D object. It is also defined as mapping or transformation of the object in projection plane or view plane. The view plane is displayed surface.




**Parallel Projection**

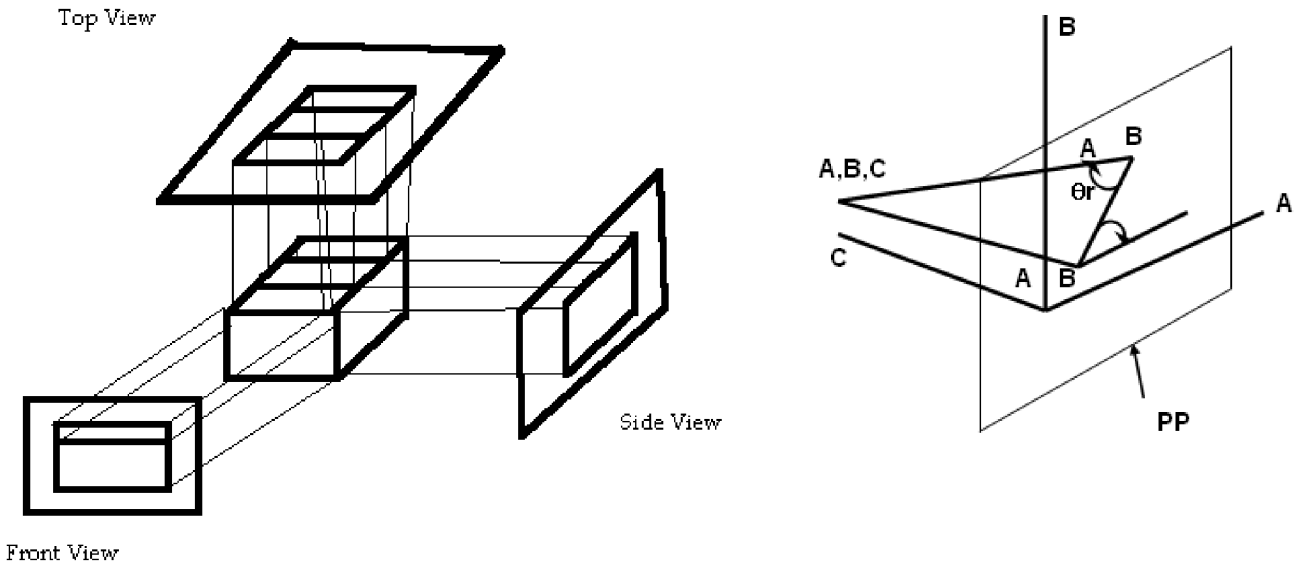
Parallel Projection use to display picture in its true shape and size. When projectors are perpendicular to view plane then is called **orthographic projection**. The parallel projection is formed by extending parallel lines from each vertex on the object until they intersect the plane of the screen. The point of intersection is the projection of vertex.

Parallel projections are used by architects and engineers for creating working drawing of the object, for complete representations require two or more views of an object using different planes.

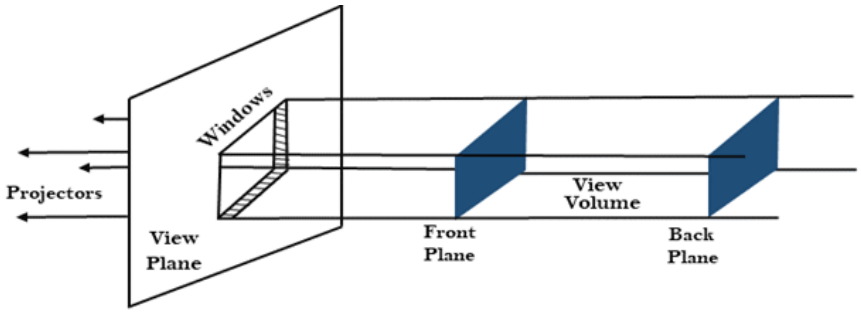


There are two types of parallel projections. They are:

1. ***Orthographic Projections:*** This is the simpler of the two parallel projections. In orthographic projections, the direction of projection is perpendicular to the projection plane. Engineering drawings often use front, side, and top orthographic views of an object. illustrates the three orthographic views of an object.




Orthographic projections that show more than one side of an object are called axonometric orthographic projections. An isometric projection is the most basic type of axonometric projection, where the projection plane intersects each co-ordinate axis in the model co-ordinate system at an equal distance. This projection is a method used to show an object in all three dimensions in a single view.

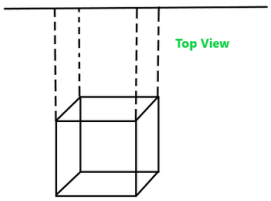



**(a) Viewing Volume in orthographic projection**

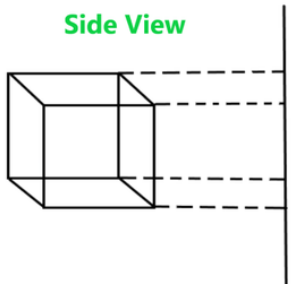
**Orthographic Projection Types**

**I. Multiview Projection : It is further divided into three categories –**

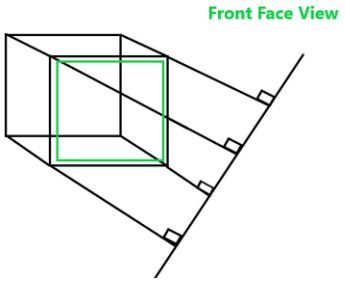
**(1) Top-View : In this projection, the rays that emerge from the top of the polygon surface are observed.**




**2) <u>Side-View</u> : It is another type of projection orthographic projection where the side view of the polygon surface is observed.**

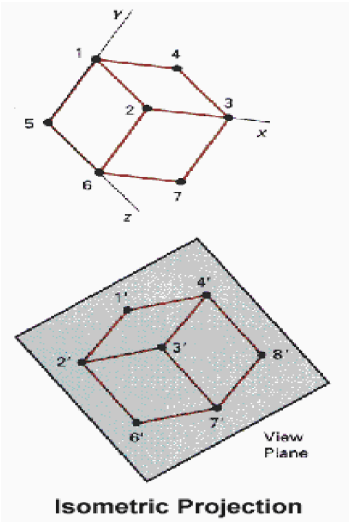



**3) <u>Front-view</u> : In this orthographic projection front face view of the object is observed.**



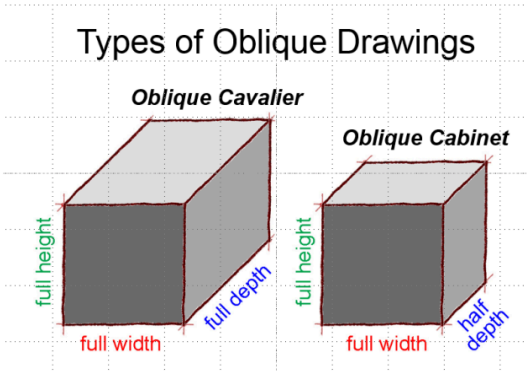

**II Axonometric : Axonometric projection is an orthographic projection, where the projection lines are perpendicular to the plane of projection, and the object is rotated around one or more of its axes to show multiple sides. It is further divided into three categories :**

1. **Isometric Projection:** All projectors make equal angles generally angle is of 30°.
2. **Dimetric:** In these two projectors have equal angles. With respect to two principle axis.
3. **Trimetric:** The direction of projection makes unequal angle with their principle axis.



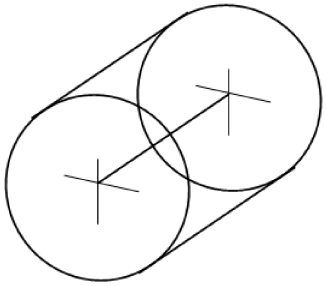
Isometric Projection

2. **Oblique Projections***:* If the direction of projection is not perpendicular to the projection plane, then it is called as oblique projection **Oblique Projection Types**
    1. **Cavalier:** All lines perpendicular to the projection plane are projected with no change in length.
    2. **Cabinet:** All lines perpendicular to the projection plane are projected to one half of their length. These give a realistic appearance of object.

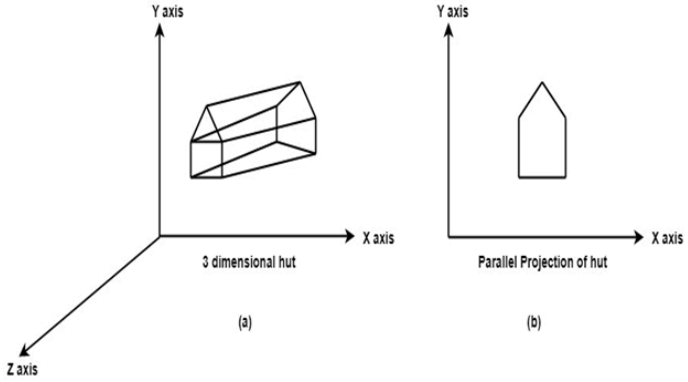



**Oblique Projections**

Oblique projection projects an image by intersecting the parallel rays from the three-dimensional source object with the drawing surface. In oblique projection, the parallel lines of the source object produce parallel lines in the projected image. The projectors produce the projected image by intersecting the projection plane at an oblique angle.



As shown in figure 10.13, oblique projection is not really a 3-D system, **it is a 2-D view of an object with forced depth.** One way to draw using an oblique view is to draw one side of the object in two dimensions that is flat. Then, draw the other sides at an angle of 45 degrees. Instead of drawing the sides full size they are only drawn with half the depth creating




forced depth - adding an element of realism to the object. Even with this forced depth, oblique drawings look very unconvincing to the eye. **For this reason, oblique projection is not often used by professional designers and engineers.**

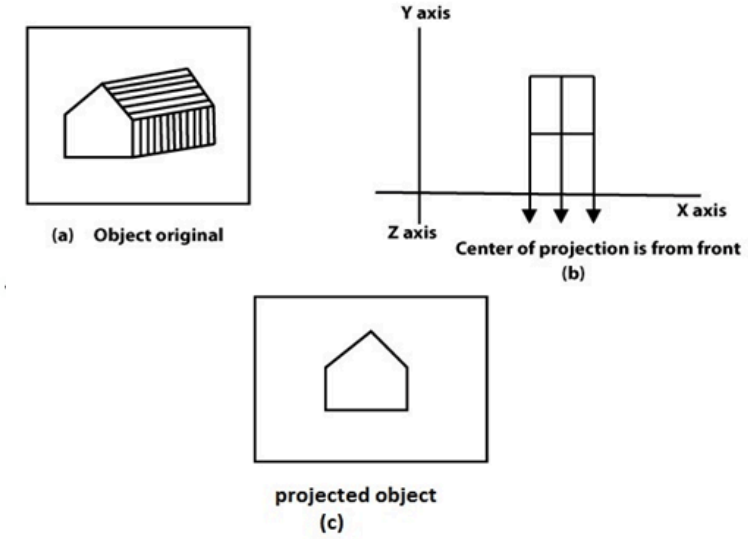
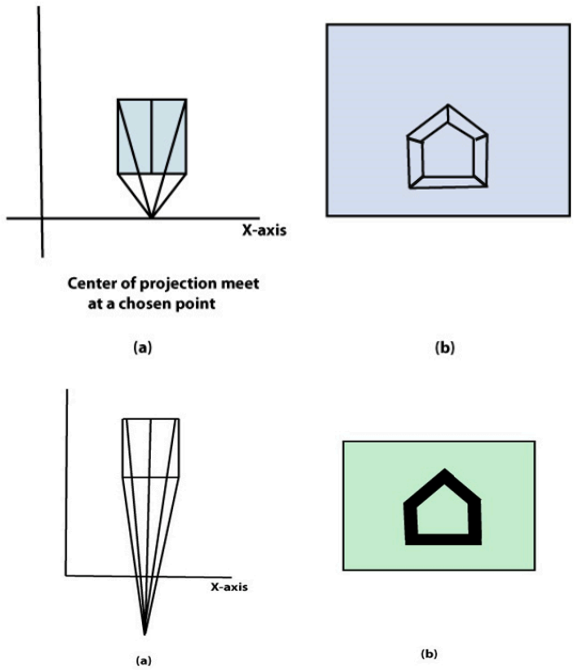
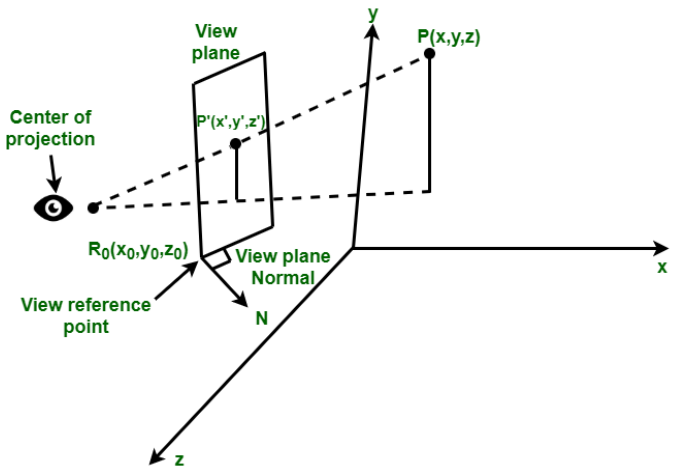



**Fig (a) shows original object. Fig (b) shows object when projection is taken. Fig (c) gives projected object.**



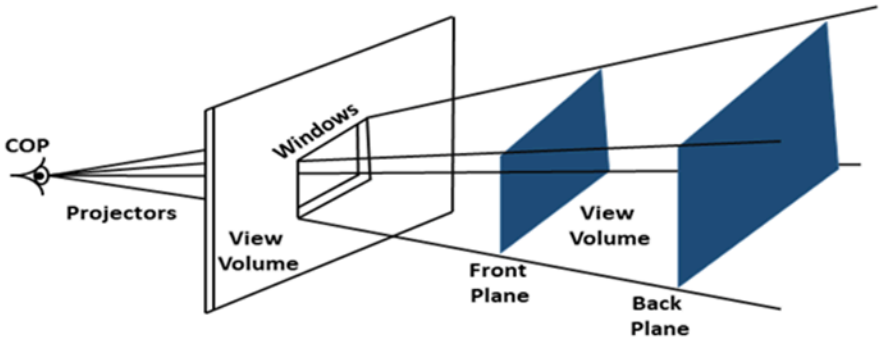

**Perspective Projection and its Types**

In Perspective Projection the **center of projection** is at finite distance from **projection plane**. This projection produces realistic views but does not preserve relative proportions of an object dimensions. Projections of distant object are smaller than projections of objects of same size that are closer to projection plane. The perspective projection can be easily described by following figure :



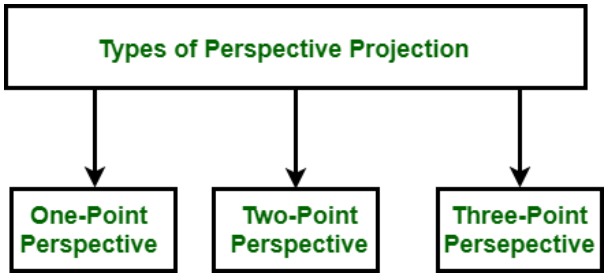

1. **Center of Projection –** It is a point where lines or projection that are not parallel to projection plane appear to meet.
2. **View Plane or Projection Plane –** The view plane is determined by :
   - View reference point $R_0(x_0, y_0, z_0)$
   - View plane normal.

3. **Location of an Object –** It is specified by a point P that is located in world coordinates at (x, y, z) location. The objective of perspective projection is to determine the image point P' whose coordinates are (x', y', z')

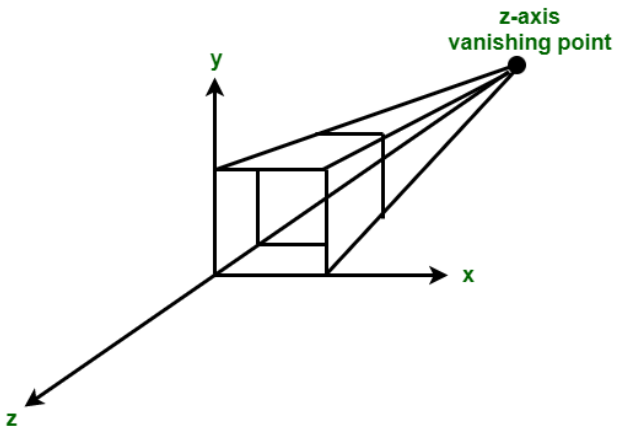



(b) Viewing volume in perspective projection

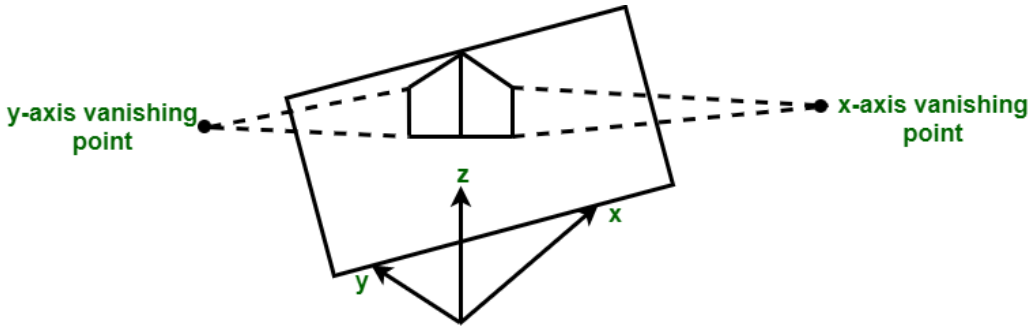
**Types of Perspective Projection :** Classification of perspective projection is on basis of vanishing points (It is a point in image where a parallel line through center of projection intersects view plane.). We can say that a vanishing point is a point where projection line intersects view plane. The classification is as follows :




**One Point Perspective Projection –** One point perspective projection occurs when any of principal axes intersects with projection plane or we can say when projection plane is perpendicular to principal axis.

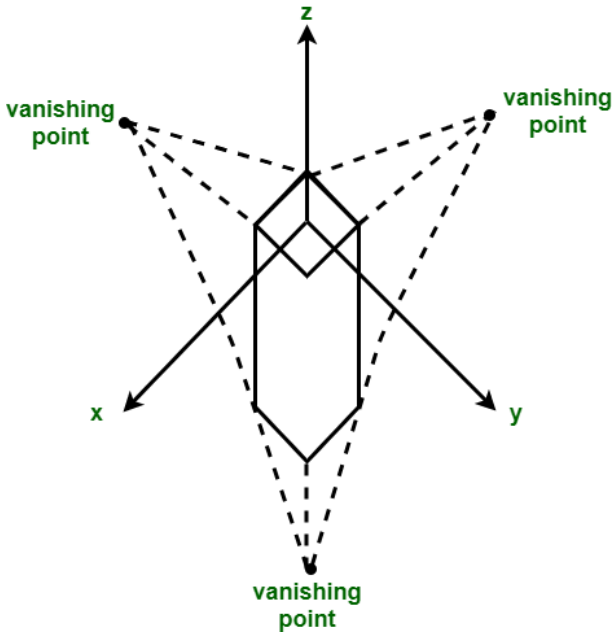


- In the above figure, z axis intersects projection plane whereas x and y axis remain parallel to projection plane.
- **Two Point Perspective Projection –** Two point perspective projection occurs when projection plane intersects two of principal axis.




In the above figure, projection plane intersects x and y axis whereas z axis remains parallel to projection plane.

- **Three Point Perspective Projection –** Three point perspective projection occurs when all three axis intersects with projection plane. There is no any principal axis which is parallel to projection

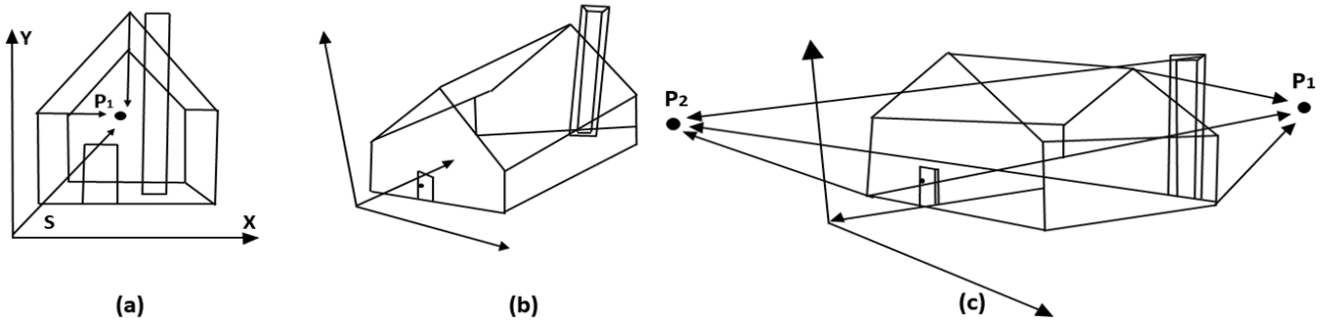



**Application of Perspective Projection :** The perspective projection technique is used by artists in preparing drawings of three-dimensional objects and scenes.

Anomalies in Perspective Projection

It introduces several anomalies due to these object shape and appearance gets affected.

1. **Perspective foreshortening:** The size of the object will be small of its distance from the center of projection increases.
2. **Vanishing Point:** All lines appear to meet at some point in the view plane.
3. **Distortion of Lines:** A range lies in front of the viewer to back of viewer is appearing to six rollers.




Foreshortening of the z-axis in fig (a) produces one vanishing point, $P_1$. Foreshortening the x and z-axis results in two vanishing points in fig (b). Adding a y-axis foreshortening in fig (c) adds vanishing point along the negative y-axis.

**Difference Between Parallel Projection And Perspective Projection :**

| SR.NO | Parallel Projection | Perspective Projection |
|---|---|---|
| 1 | Parallel projection represents the object in a different way like telescope. | Perspective projection represents the object in three dimensional way. |
| 2 | In parallel projection, these effects are not created. | In perspective projection, objects that are far away appear smaller, and objects that are near appear bigger. |
| 3 | The distance of the object from the center of projection is infinite. | The distance of the object from the center of projection is finite. |
| 4 | Parallel projection can give the accurate view of object. | Perspective projection cannot give the accurate view of object. |
| 5 | The lines of parallel projection are parallel. | The lines of perspective projection are not parallel. |

| SR.NO | Parallel Projection | Perspective Projection |
|---|---|---|
| 6 | Projector in parallel projection is parallel. | Projector in perspective projection is not parallel. |
| 7 | Two types of parallel projection :<br>1.Orthographic,<br>2.Oblique | Three types of perspective projection:<br>1.one point perspective,<br>2.Two point perspective,<br>3. Three point perspective, |
| 8 | It does not form realistic view of object. | It forms a realistic view of object. |

## HIDDEN SURFACE REMOVAL

When we view a picture containing non-transparent objects and surfaces, then we cannot see those objects from view which are behind from objects closer to eye. We must remove these hidden surfaces to get a realistic screen image. The identification and removal of these surfaces is called **Hidden-surface problem**.

There are two approaches for removing hidden surface problems − **Object-Space method** and **Image-space method**. The Object-space method is implemented in physical coordinate system and image-space method is implemented in screen coordinate system.

When we want to display a 3D object on a 2D screen, we need to identify those parts of a screen that are visible from a chosen viewing position.

**Hidden Surface Removal**

1. One of the most challenging problems in computer graphics is the removal of hidden parts from images of solid objects.
2. In real life, the opaque material of these objects obstructs the light rays from hidden parts and prevents us from seeing them.
3. In the computer generation, no such automatic elimination takes place when objects are projected onto the screen coordinate system.
4. Instead, all parts of every object, including many parts that should be invisible are displayed.
5. To remove these parts to create a more realistic image, we must apply a hidden line or hidden surface algorithm to set of objects.
6. The algorithm operates on different kinds of scene models, generate various forms of output or cater to images of different complexities.
7. All use some form of geometric sorting to distinguish visible parts of objects from those that are hidden.
8. Just as alphabetical sorting is used to differentiate words near the beginning of the alphabet from those near the ends.
9. Geometric sorting locates objects that lie near the observer and are therefore visible.

10. Hidden line and Hidden surface algorithms capitalize on various forms of coherence to reduce the computing required to generate an image.
11. Different types of coherence are related to different forms of order or regularity in the image.
12. Scan line coherence arises because the display of a scan line in a raster image is usually very similar to the display of the preceding scan line.
13. **Frame coherence** in a sequence of images designed to show motion recognizes that successive frames are very similar.
14. **Object coherence** results from relationships between different objects or between separate parts of the same objects.
15. A hidden surface algorithm is generally designed to exploit one or more of these coherence properties to increase efficiency.
16. Hidden surface algorithm bears a strong resemblance to two-dimensional scan conversions.

Types of hidden surface detection algorithms

1. Object space methods
2. Image space methods

**Object space methods:** In this method, various parts of objects are compared. After comparison visible, invisible or hardly visible surface is determined. These methods generally decide visible surface. In the wireframe model, these are used to determine a visible line. So these algorithms are line based instead of surface based. Method proceeds by determination of parts of an object whose view is obstructed by other object and draws these parts in the same color.

**Image space methods:** Here positions of various pixels are determined. It is used to locate the visible surface instead of a visible line. Each point is detected for its visibility. If a point is visible, then the pixel is on, otherwise off. So the object close to the viewer that is pierced by a projector through a pixel is determined. That pixel is drawn is appropriate color.

These methods are also called a **Visible Surface Determination**. The implementation of these methods on a computer requires a lot of processing time and processing power of the computer.

The image space method requires more computations. Each object is defined clearly. Visibility of each object surface is also determined.

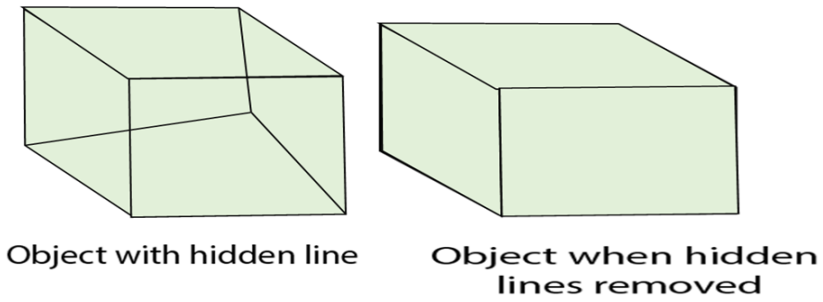
Differentiate between Object space and Image space method

| Object Space | Image Space |
|---|---|
| 1. Image space is object based. It concentrates on geometrical relation among objects in the scene. | 1. It is a pixel-based method. It is concerned with the final image, what is visible within each raster pixel. |

| | |
|---|---|
| 2. Here surface visibility is determined. | 2. Here line visibility or point visibility is determined. |
| 3. It is performed at the precision with which each object is defined, No resolution is considered. | 3. It is performed using the resolution of the display device. |
| 4. Calculations are not based on the resolution of the display so change of object can be easily adjusted. | 4. Calculations are resolution base, so the change is difficult to adjust. |
| 5. These were developed for vector graphics system. | 5. These are developed for raster devices. |
| 6. Object-based algorithms operate on continuous object data. | 6. These operate on object data. |
| 7. Vector display used for object method has large address space. | 7. Raster systems used for image space methods have limited address space. |
| 8. Object precision is used for application where speed is required. | 8. There are suitable for application where accuracy is required. |
| 9. It requires a lot of calculations if the image is to be enlarged. | 9. Image can be enlarged without losing accuracy. |
| 10. If the number of objects in the scene increases, computation time also increases. | 10. In this method complexity increase with the complexity of visible parts. |

Similarity of object and Image space method

In both method sorting is used a depth comparison of individual lines, surfaces are objected to their distances from the view plane.




**Considerations for selecting or designing hidden surface algorithms:** Following three considerations are taken:

Algorithms used for hidden line surface detection

1. Back Face Removal Algorithm
2. Z-Buffer Algorithm
3. Depth- Sort (Painter Algorithm)
4. Scan Line Method
5. Area Subdivision Method

**Back Face Removal Algorithm**

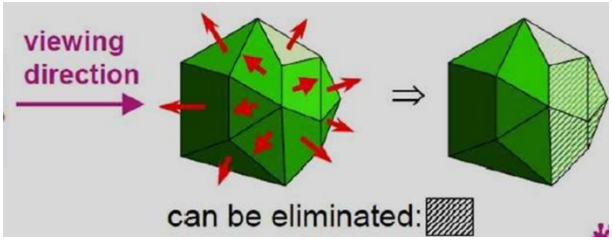
When we project 3-D objects on a 2-D screen, we need to detect the faces that are hidden on 2D.

**Back-Face detection**, also known as **Plane Equation method,** is an object space method in which objects and parts of objects are compared to find out the visible surfaces. Let us consider a polygon surface whose visibility needs to decided. The idea is to check if the polygon will be facing away from the viewer or not. If it does so, discard it for the current frame and move onto the next one. Each surface has a normal vector. If this normal vector is pointing in the direction of the center of projection, then it is a front face and can be seen by the viewer. If this normal vector is pointing away from the center of projection, then it is a back face and cannot be seen by the viewer.

So we use back face removal algo such that:

- It is used to plot only surfaces which will face the camera. The objects on the back side are not visible.
- This method will remove 50% of polygons from the scene if the parallel projection is used. If the perspective projection is used, then more than 50% of the invisible area will be removed.
- The object is nearer to the center of projection, number of polygons from the back will be removed.
- It applies to individual objects. It does not consider the interaction between various objects. Many polygons are obscured by front faces, although they are closer to the viewer, so for removing such faces back face removal algorithm is used.
- When the projection is taken, any projector ray from the center of projection through viewing screen to object pieces at two points, one is visible front surfaces, and another is not visible back surface.
- This algorithm acts a preprocessing step for another algorithm. The back face algorithm can be represented geometrically. Each polygon has several vertices.
- All vertices are numbered in clockwise. The normal $N_1$ is generated a cross product of any two successive edge vectors. $N_1$ represent vector perpendicular to face and point outward from polyhedron surface

$N_1=(v_2-v_1)(v_3-v_2)$
If $N_1.P\geq 0$ visible
$N_1.P<0$ invisible




Hence, it's a fast and simple object-space method for identifying the back faces of a polyhedron is based on the "inside-outside" tests. A point x,y,z is "inside" a polygon surface with plane parameters A, B, C, and D if When an inside point is along the line of sight to the surface, the polygon must be a back face , We can simplify this test by considering the normal vector **N** to a polygon surface, which has Cartesian components A,B,C.

**Back Face Removal Algorithm**

Repeat for all polygons in the scene.

1. Do numbering of all polygons in clockwise direction i.e.
   $v_1$ $v_2$ $v_3$.....$v_z$
2. Calculate normal vector i.e. $N_1$
   $N_1=(v_2-v_1)*(v_3-v_2)$
3. Consider projector P, it is projection from any vertex
   Calculate dot product
   Dot=N.P
4. Test and plot whether the surface is visible or not.
   If Dot $\geq 0$ then
   surface is visible
   else
   Not visible

**Advantage**

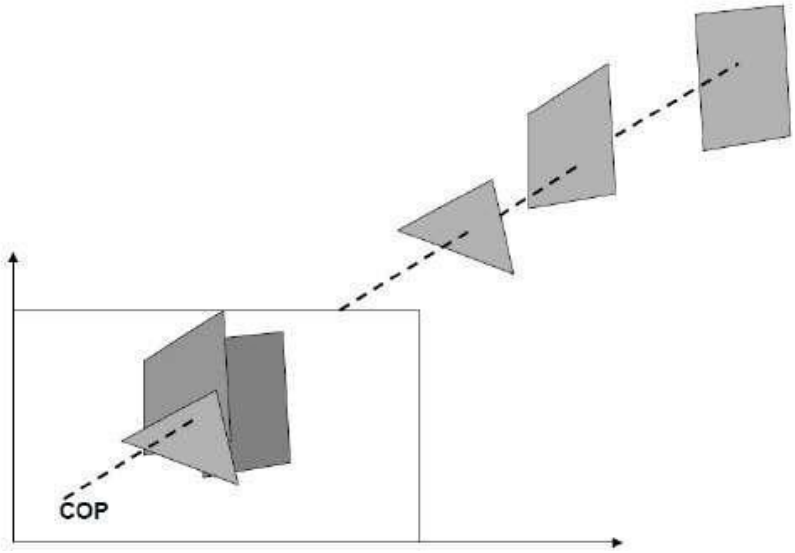
1. It is a simple and straight forward method.
2. It reduces the size of databases, because no need of store all surfaces in the database, only the visible surface is stored.

**Depth Buffer Z−Buffer Method**

This method is developed by Cutmull. It is an image-space approach. The basic idea is to test the Z-depth of each surface to determine the closest visible surface. In this method each surface is processed separately one-pixel position at a time across the surface. The depth values for a pixel are compared and the closest smallest z surface determines the color to be displayed in the frame buffer.

It is applied very efficiently on surfaces of polygon. Surfaces can be processed in any order. To override the closer polygons from the far ones, two buffers named **frame buffer** and **depth buffer,** are used.
**Depth buffer** is used to store depth values for x, y position, as surfaces are processed $0 \leq depth \leq 1$.
The **frame buffer** is used to store the intensity value of color value at each position x,y.

The z-coordinates are usually normalized to the range [0, 1]. The 0 value for z-coordinate indicates back clipping pane and 1 value for z-coordinates indicates front clipping pane.




**Algorithm**

**Step-1** − Set the buffer values −
Depthbuffer x,y = 0
Framebuffer x,y = background color

**Step-2** − Process each polygon One at a time
For each projected x,y pixel position of a polygon, calculate depth z.
If Z > depthbuffer x,y

Compute surface color,
set depthbuffer x,y = z,
framebuffer x,y = surfacecolor x,y

CODE

```
Step 1: Initialize the depth of each pixel.
        d(i, j) = infinite (max length)
        Initialize the color value for each pixel as
        c(i, j) = background color
Step 2: for each polygon, do the following steps :
        for (each pixel in polygon's projection)
                {
                  find depth i.e, z of polygon at (x, y) corresponding to pixel (i, j)
                        if (z < d(i, j))
                        {
                              d(i, j) = z;
                              c(i, j) = color;
}}
```

Let's denote the depth at point A as Z and at point B as Z'. Therefore, using equation of plane:

$AX + BY + CZ + D = 0$ implies

$$Z = (-AX - BY - D)/C \quad \text{----------(1)}$$

Similarly, $$Z' = (-A(X + 1) - BY - D)/C \quad \text{----------(2)}$$

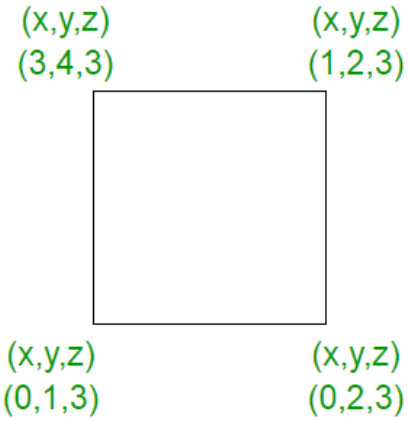
Hence from (1) and (2), we conclude :

$$Z' = Z - A/C \quad \text{----------(3)}$$

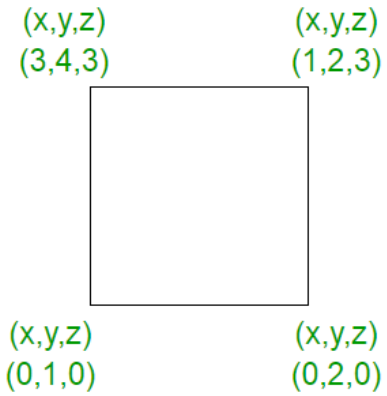
Hence, calculation of depth can be done by recording the plane equation of each polygon in the (normalized) viewing coordinate system and then using the incremental method to find the depth Z.
So, to summarize, it can be said that this approach compares surface depths at each pixel position on the projection plane. Object depth is usually measured from the view plane along the z-axis of a viewing system.

EXAMPLE

Let's consider an example to understand the algorithm in a better way. Assume the polygon given is as below :




- In starting, assume that the depth of each pixel is infinite.
- As the z value i.e, the depth value at every place in the given polygon is 3, on applying the algorithm.
  - Now, let's change the z values. In the figure given below, the z values goes from 0 to 3.

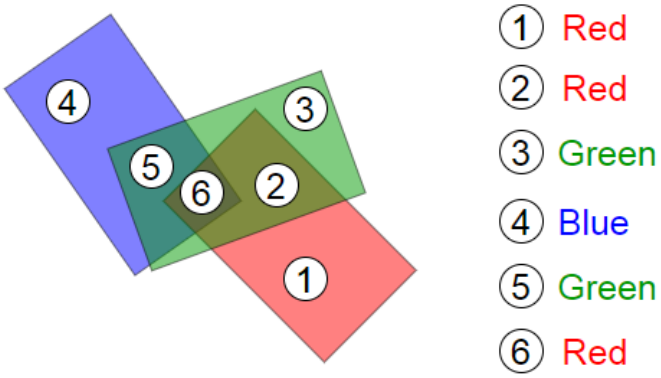



Therefore, in the Z buffer method, each surface is processed separately one position at a time across the surface. After that the depth values i.e, the z values for a pixel are compared and the closest i.e, (smallest z) surface determines the color to be displayed in frame buffer. The z values, i.e, the depth values are

usually normalized to the range [0, 1]. When the z = 0, it is known as ***Back Clipping Pane*** and when z = 1, it is called as the ***Front Clipping Pane***.
In this method, 2 buffers are used :

1. Frame buffer
2. Depth buffer

**Points to remember :**
**1)** Z buffer method does not require pre-sorting of polygons.
**2)** This method can be executed quickly even with many polygons.
**3)** This can be implemented in hardware to overcome the speed problem.
**4)** No object to object comparison is required.
**5)** This method can be applied to non-polygonal objects.
**6)** Hardware implementation of this algorithm are available in some graphics workstations.
**7)** The method is simple to use and does not require additional data structure.
**8)** The z-value of a polygon can be calculated incrementally.
**9)** Cannot be applied on transparent surfaces i.e, it only deals with opaque surfaces. For ex




**10)** If only a few objects in the scene are to be rendered, then this method is less attractive because of additional buffer and the overhead involved in updating the buffer.

**11)** Wastage of time may occur because of drawing of hidden objects.

Advantages

- It is easy to implement.
- It reduces the speed problem if implemented in hardware.
- It processes one object at a time.

Disadvantages

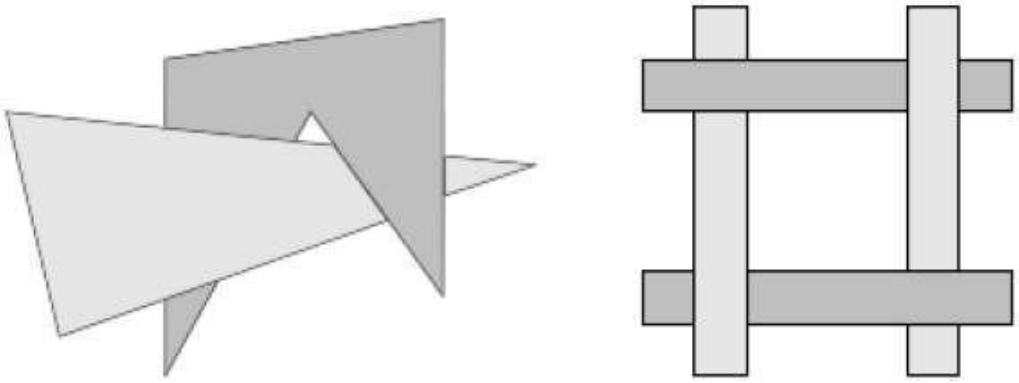
- The depth buffer Algorithm is not always practical because of the enormous size of depth and intensity arrays.
- It requires large memory. Generating an image with a raster of 500 x 500 pixels requires 2, 50,000 storage locations for each array.
- It is time consuming process.

**Depth Sorting Method (Painter Algorithm)**

Depth sorting method uses both image space and object-space operations. The depth-sorting method performs two basic functions −

- First, the surfaces are sorted in order of decreasing depth.
- Second, the surfaces are scan-converted in order, starting with the surface of greatest depth.

The scan conversion of the polygon surfaces is performed in image space. This method for solving the hidden-surface problem is often referred to as the **painter's algorithm**. The following figure shows the effect of depth sorting −



The algorithm begins by sorting by depth. For example, the initial "depth" estimate of a polygon may be taken to be the closest z value of any vertex of the polygon.

Let us take the polygon P at the end of the list. Consider all polygons Q whose z-extents overlap P's. Before drawing P, we make the following tests. If any of the following tests is positive, then we can assume P can be drawn before Q.

- Do the x-extents not overlap?
- Do the y-extents not overlap?
- Is P entirely on the opposite side of Q's plane from the viewpoint?
- Is Q entirely on the same side of P's plane as the viewpoint?
- Do the projections of the polygons not overlap?

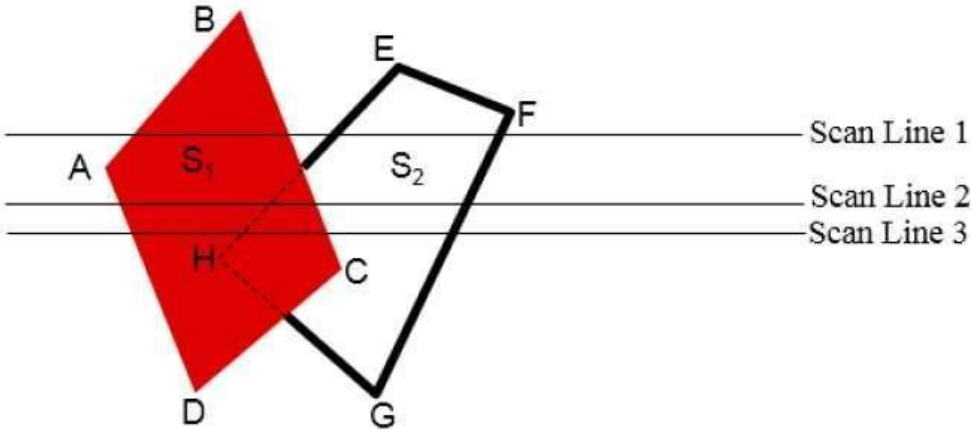
If all the tests fail, then we split either P or Q using the plane of the other. The new cut polygons are inserting into the depth order and the process continues. Theoretically, this partitioning could generate $O(n^2)$ individual polygons, but in practice, the number of polygons is much smaller.

**Scan-Line Method**

It is an image-space method to identify visible surface. This method has a depth information for only single scan-line. In order to require one scan-line of depth values, we must group and process all polygons intersecting a given scan-line at the same time before processing the next scan-line. Two important tables, **edge table** and **polygon table,** are maintained for this.

**The Edge Table** − It contains coordinate endpoints of each line in the scene, the inverse slope of each line, and pointers into the polygon table to connect edges to surfaces.

**The Polygon Table** − It contains the plane coefficients, surface material properties, other surface data, and may be pointers to the edge table.




To facilitate the search for surfaces crossing a given scan-line, an active list of edges is formed. The active list stores only those edges that cross the scan-line in order of increasing x. Also a flag is set for each surface to indicate whether a position along a scan-line is either inside or outside the surface.

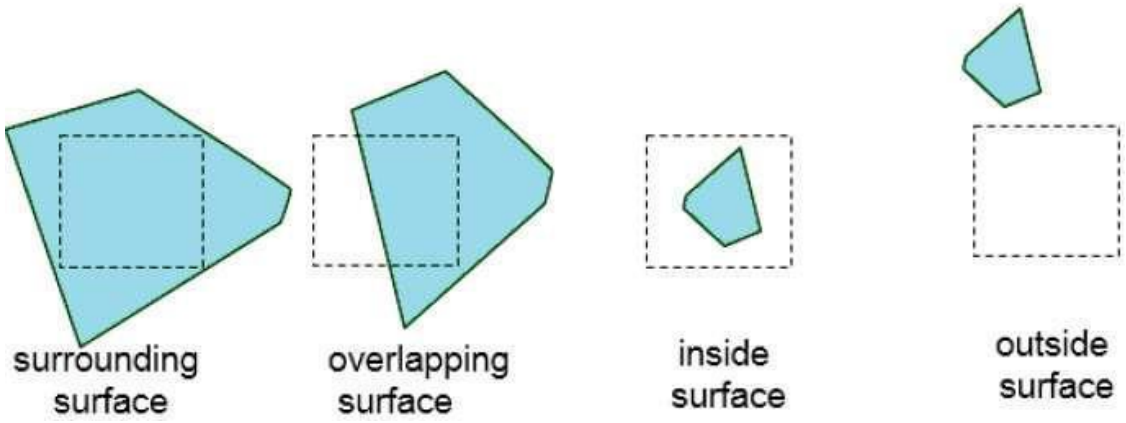
Pixel positions across each scan-line are processed from left to right. At the left intersection with a surface, the surface flag is turned on and at the right, the flag is turned off. You only need to perform depth calculations when multiple surfaces have their flags turned on at a certain scan-line position.

**Area-Subdivision Method**

The area-subdivision method takes advantage by locating those view areas that represent part of a single surface. Divide the total viewing area into smaller and smaller rectangles until each small area is the projection of part of a single visible surface or no surface at all.

Continue this process until the subdivisions are easily analyzed as belonging to a single surface or until they are reduced to the size of a single pixel. An easy way to do this is to successively divide the area into four equal parts at each step. There are four possible relationships that a surface can have with a specified area boundary.

- **Surrounding surface** − One that completely encloses the area.
- **Overlapping surface** − One that is partly inside and partly outside the area.
- **Inside surface** − One that is completely inside the area.
- **Outside surface** − One that is completely outside the area.



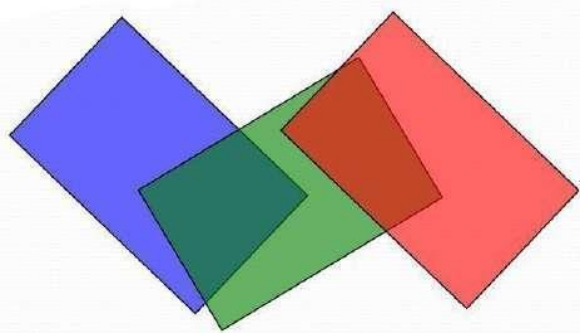

The tests for determining surface visibility within an area can be stated in terms of these four classifications. No further subdivisions of a specified area are needed if one of the following conditions is true −

- All surfaces are outside surfaces with respect to the area.
- Only one inside, overlapping or surrounding surface is in the area.
- A surrounding surface obscures all other surfaces within the area boundaries.

**A-Buffer Method**

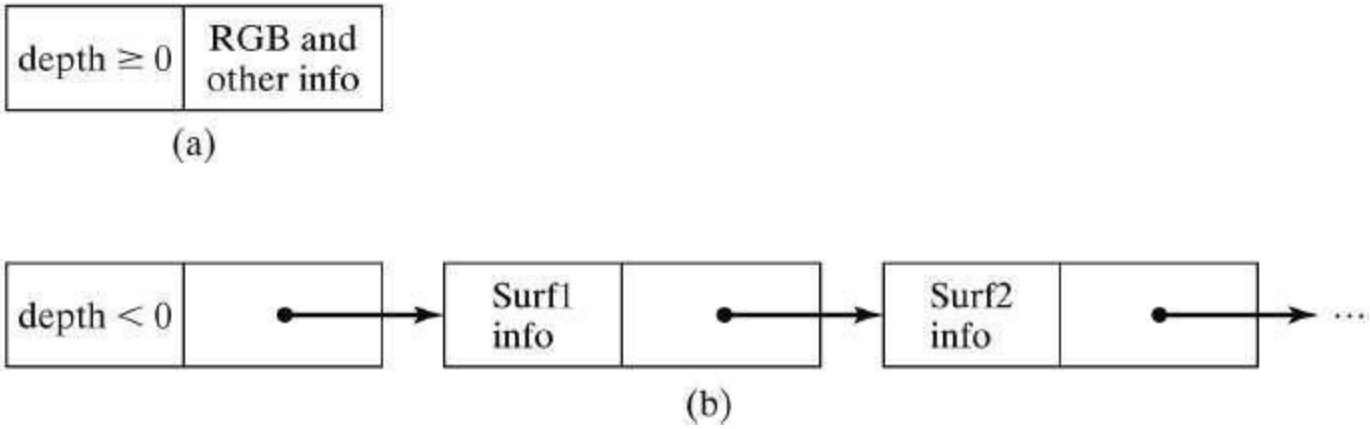
The A-buffer method is an extension of the depth-buffer method. The A-buffer method is a visibility detection method developed at Lucas film Studios for the rendering system Renders Everything You Ever Saw REYES.

The A-buffer expands on the depth buffer method to allow transparencies. The key data structure in the A-buffer is the accumulation buffer.



Each position in the A-buffer has two fields −

- **Depth field** − It stores a positive or negative real number
- **Intensity field** − It stores surface-intensity information or a pointer value




If depth >= 0, the number stored at that position is the depth of a single surface overlapping the corresponding pixel area. The intensity field then stores the RGB components of the surface color at that point and the percent of pixel coverage.

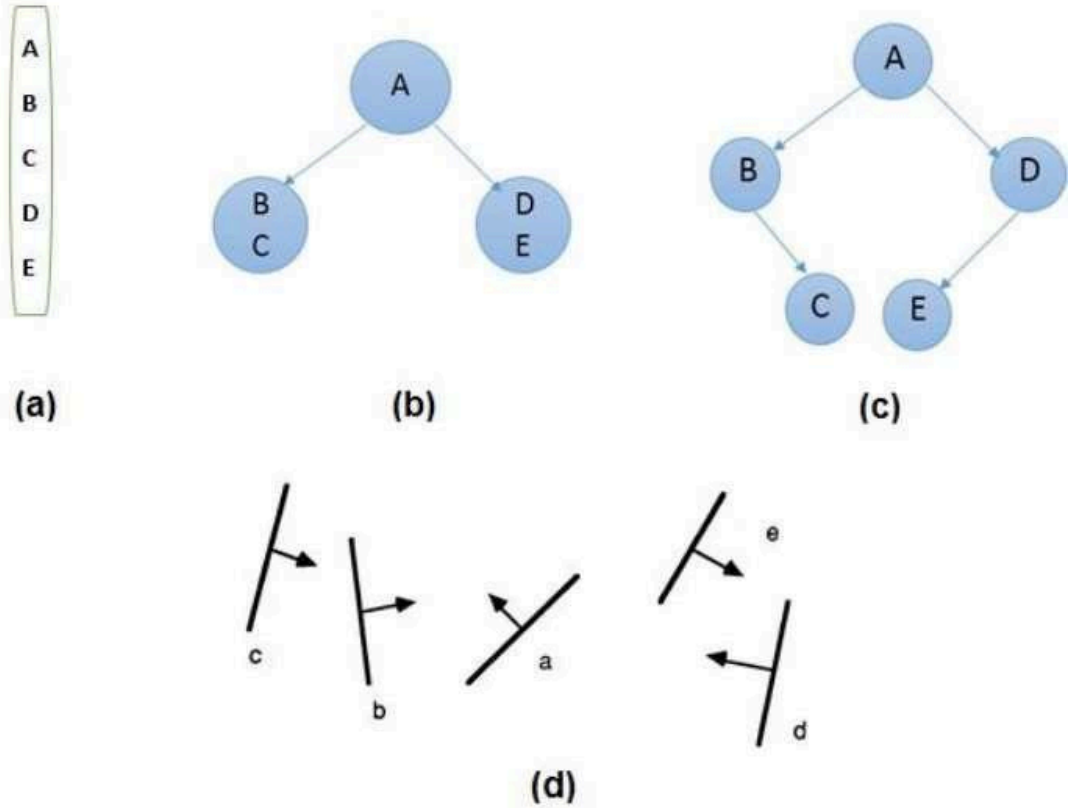
If depth < 0, it indicates multiple-surface contributions to the pixel intensity. The intensity field then stores a pointer to a linked list of surface data. The surface buffer in the A-buffer includes −

- RGB intensity components
- Opacity Parameter
- Depth
- Percent of area coverage
- Surface identifier

The algorithm proceeds just like the depth buffer algorithm. The depth and opacity values are used to determine the final color of a pixel.

**Binary Space Partition BSP Trees**

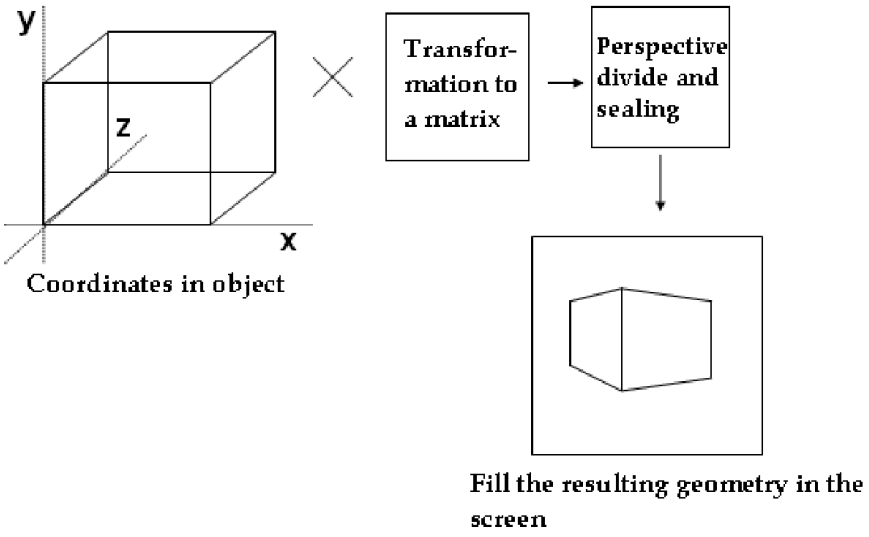
Binary space partitioning is used to calculate visibility. To build the BSP trees, one should start with polygons and label all the edges. Dealing with only one edge at a time, extend each edge so that it splits the plane in two. Place the first edge in the tree as root. Add subsequent edges based on whether they are inside or outside. Edges that span the extension of an edge that is already in the tree are split into two and both are added to the tree.




- From the above figure, first take **A** as a root.
- Make a list of all nodes in figure a.
- Put all the nodes that are in front of root **A** to the left side of node **A** and put all those nodes that are behind the root **A** to the right side as shown in figure b.
- Process all the front nodes first and then the nodes at the back.
- As shown in figure c, we will first process the node **B**. As there is nothing in front of the node **B**, we have put NIL. However, we have node **C** at back of node **B**, so node **C** will go to the right side of node **B**.
- Repeat the same process for the node **D**.

**Rasterization**

Rasterization, also known as **Scan line rendering,** is a process where the outline representation of a 3-D object is captured, which is usually a triangular mesh. The captured representation is then transformed into screen space, and projected on the screen. The resulting view is obtained in 2-D geometry as shown cin figure below. This technique is employed in various computer games and other interactive visualizations since these modern 3-D graphics hardware can rasterize at a fast speed.




Now, let us discuss some of the prime differences between ray tracing and rasterization techniques.

**Ray tracing vs. Rasterization**

- The speed of ray tracing with proper acceleration structures is O (log n) to the number of primitives in the scene. Thus, it is a little bit slower.
  Rasterization is usually O (n). Thus, it is faster than Ray tracing.
- Effects such as reflections, shadows, and global illumination can be easily added by simply shooting more rays in smart ways.
  These effects find it much harder or even impossible to take effect appropriately through rasterization.
- Usually very low level of memory coherency is achieved through ray tracing.
  Extremely high level of memory coherency can be achieved through rasterization.
- While ray tracing, the entire picture has to be kept in memory.
  While rasterizing, the entire picture need not to be kept in memory as it can be drawn piece by piece